

Heidelberg University

Faculty of Mathematics and Computer Science

Institute of Computer Science

Artificial Intelligence for Programming (AIP)

Bachelor's Thesis

Comparison of LoRA and Knowledge Editing for Improving Neural Decompilation

Name: Hendrik Lohmar
Supervisor: Prof. Dr. Artur Andrzejak
Date of Submission: January 16, 2026

Abstract

English

Decompilation, the process of recovering source code from compiled binaries, is essential for security analysis and legacy software maintenance. Traditional decompilers like Ghidra produce pseudocode that aids human understanding but often cannot be directly compiled or executed. This thesis investigates parameter-efficient methods for adapting Large Language Models to refine Ghidra pseudocode into valid, compilable C code.

We compare two fundamentally different approaches: Low-Rank Adaptation (LoRA), which trains small adapter matrices while keeping base model weights frozen, and Knowledge Editing, which surgically modifies specific model weights to correct individual errors. We evaluate these methods using CodeLlama-7B-Instruct on a benchmark of 151 functions with functional correctness tests.

Our experiments reveal that LoRA fine-tuning substantially improves decompilation quality. General fine-tuning achieves 84.33% compile rate and 23.95% functional correctness (baseline: 18.94% and 15.50%). Mixed training combining error-specific examples with general data reaches the highest functional correctness of 28.08%, though at a reduced compile rate of 64.50%. Knowledge Editing, however, proved ineffective for this task, as decompilation errors involve complex structural transformations rather than discrete factual corrections.

A key finding is the syntactic-semantic gap: high compile rates do not guarantee functional correctness. Models can learn to produce syntactically valid code while still generating semantically incorrect implementations. This gap highlights the importance of functional testing over surface-level metrics in neural decompilation research.

Abstract

Deutsch

Dekompilierung, der Prozess der Wiederherstellung von Quellcode aus kompilierten Binärdateien, ist für Sicherheitsanalysen und die Wartung von Legacy-Software unerlässlich. Traditionelle Decompiler wie Ghidra erzeugen Pseudocode, der das menschliche Verständnis unterstützt, aber oft nicht direkt kompiliert oder ausgeführt werden kann. Diese Arbeit untersucht parametereffiziente Methoden zur Anpassung von Large Language Models, um Ghidra-Pseudocode in gültigen, kompilierbaren C-Code zu transformieren.

Wir vergleichen zwei grundlegend verschiedene Ansätze: Low-Rank Adaptation (LoRA), das kleine Adaptermatrizen trainiert, während die Basismodellgewichte eingefroren bleiben, und Knowledge Editing, das gezielt spezifische Modellgewichte modifiziert, um einzelne Fehler zu korrigieren. Wir evaluieren diese Methoden mit CodeLlama-7B-Instruct auf einem Benchmark von 151 Funktionen mit funktionalen Korrektheitstests.

Unsere Experimente zeigen, dass LoRA-Fine-Tuning die Dekompilierungsqualität erheblich verbessert. Allgemeines Fine-Tuning erreicht 84,33% Kompilierungsrate und 23,95% funktionale Korrektheit (Baseline: 18,94% und 15,50%). Gemischtes Training, das fehlerspezifische Beispiele mit allgemeinen Daten kombiniert, erzielt die höchste funktionale Korrektheit von 28,08%, jedoch bei einer reduzierten Kompilierungsrate von 64,50%. Knowledge Editing erwies sich jedoch als ineffektiv für diese Aufgabe, da Dekompilierungsfehler komplexe strukturelle Transformationen erfordern, nicht diskrete faktische Korrekturen.

Eine zentrale Erkenntnis ist die syntaktisch-semantische Lücke: Hohe Kompilierungsraten garantieren keine funktionale Korrektheit. Modelle können lernen, syntaktisch gültigen Code zu erzeugen, der dennoch semantisch falsche Implementierungen enthält. Diese Lücke unterstreicht die Bedeutung funktionaler Tests gegenüber oberflächlichen Metriken in der Forschung zur neuronalen Dekompilierung.

Acknowledgments

The author acknowledges support by the state of Baden-Württemberg through bwHPC.

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Research Questions	2
1.3	Contributions	2
1.4	Thesis Outline	2
2	Background	4
2.1	Decompilation Fundamentals	4
2.1.1	Compilation and Information Loss	4
2.1.2	Ghidra Decompiler	4
2.2	Transformer Architecture	6
2.2.1	Self-Attention Mechanism	7
2.2.2	Feed-Forward Layers	7
2.2.3	Layer Composition	8
2.3	Large Language Models	9
2.3.1	Pretraining and Scale	9
2.3.2	LLMs for Code	9
2.4	Neural Decompilation	10
2.5	Parameter-Efficient Fine-Tuning	10
2.5.1	The Challenge of Full Fine-Tuning	10
2.5.2	Low-Rank Adaptation (LoRA)	11
2.5.3	QLoRA	11
2.6	Knowledge Editing	11
2.7	Evaluation Metrics	12
3	Related Work	13
3.1	Neural Decompilation Systems	13
3.2	Research Gap and Positioning	13
4	Methodology	15
4.1	Overview	15
4.2	Data Pipeline	16
4.3	LoRA Fine-Tuning Approach	17
4.3.1	Base Model Selection	17
4.3.2	Training Configuration	17
4.3.3	Hyperparameter Search	19
4.4	Knowledge Editing Approach	20
4.4.1	Formulating Decompilation as Knowledge Editing	20
4.4.2	Implementation	20
4.4.3	Fundamental Limitations	20
4.5	Error Analysis	21
4.5.1	Evaluation Framework	21
4.5.2	Failure Categorization	22
4.5.3	Semantic Error Taxonomy	22
4.5.4	Trainability Assessment	22

4.6	Comparison Strategy	22
5	Results	24
5.1	Baseline Performance	24
5.2	LoRA Fine-Tuning Results	24
5.2.1	Result Stability	26
5.3	Error Analysis	26
5.3.1	Semantic Error Patterns	26
5.3.2	Trainability Assessment	27
5.4	Error-Specific Fine-Tuning Results	28
5.4.1	Synthetic Error Data Generation	28
5.4.2	Targeted Error Data	28
5.4.3	Mixed Training Data	28
5.4.4	Summary	29
5.5	Knowledge Editing Results	29
6	Discussion	31
6.1	Interpretation of Results	31
6.2	The Syntactic-Semantic Gap	31
6.3	Comparison with State-of-the-Art	32
6.4	Implications for Neural Decompilation	32
6.5	Limitations	33
7	Conclusion	34
7.1	Summary of Findings	34
7.2	Contributions	34
7.3	Future Work	35
	Bibliography	36

1 Introduction

Reverse engineering of software binaries is essential for security analysis, malware detection, legacy system maintenance, and vulnerability research. At the core of this process lies decompilation: transforming compiled machine code back into human-readable source code. However, decompilation is inherently lossy; the compilation process discards variable names, type information, comments, and high-level structure, leaving decompilers to reconstruct these elements through heuristic analysis [1].

Modern decompilers such as Ghidra [2] produce pseudocode that, while logically equivalent to the original program, often contains artifacts that make it difficult to read, modify, or recompile. These artifacts include non-standard type annotations (e.g., `undefined4`), synthesized variable names, and unconventional control flow constructs. Human analysts must manually refine this output, a time-consuming process that scales poorly with the volume of software requiring analysis.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in code understanding and generation [3], suggesting their potential application to decompilation refinement. Rather than replacing traditional decompilers, LLMs could serve as a post-processing step, transforming pseudocode into clean, idiomatic, and compilable C code. However, applying pre-trained LLMs directly to this task yields poor results; the models reproduce decompiler artifacts rather than translating them into standard constructs.

This thesis investigates two adaptation approaches for improving LLM-based decompilation: Low-Rank Adaptation (LoRA) [4], a parameter-efficient fine-tuning method, and Knowledge Editing [5], a technique for surgically modifying model weights to correct specific factual associations. These approaches represent fundamentally different hypotheses about the nature of decompilation errors and offer complementary strategies for addressing them.

1.1 Problem Statement

Pre-trained Large Language Models for code, such as CodeLlama [3], possess extensive knowledge of programming languages and can generate syntactically correct code in various contexts. However, when presented with Ghidra pseudocode, these models frequently reproduce decompiler-specific artifacts rather than translating them into standard C constructs. This results in output that fails to compile or, when it does compile, produces functionally incorrect results.

The central challenge is adapting these models to the decompilation task efficiently. Full fine-tuning requires updating billions of parameters, demanding significant computational resources and risking catastrophic forgetting [6] of the model’s general capabilities. Two alternative approaches warrant investigation:

Low-Rank Adaptation (LoRA) [4] introduces small trainable matrices into the model architecture, enabling task-specific adaptation while keeping the base model frozen. This

approach hypothesizes that decompilation can be learned as a general skill through exposure to pseudocode-to-source-code examples.

Knowledge Editing [5] directly modifies specific model parameters to correct targeted factual associations. This approach hypothesizes that decompilation errors stem from incorrect or missing factual mappings (e.g., that `undefined4` should map to `int`) that can be surgically corrected.

1.2 Research Questions

This thesis addresses the following research questions:

1. **RQ1:** How effectively can LoRA fine-tuning improve the functional correctness of LLM-generated decompiled code?
2. **RQ2:** Can Knowledge Editing techniques correct systematic decompiler artifacts in model output?
3. **RQ3:** What types of decompilation errors are addressable through model adaptation, and what limitations remain?
4. **RQ4:** How do error-specific training approaches compare to general fine-tuning for improving decompilation quality?

1.3 Contributions

This thesis makes the following contributions:

1. A systematic evaluation of LoRA fine-tuning for neural decompilation, demonstrating a 54% relative improvement in functional correctness (Pass@1 from 15.50% to 23.95%) and a 4.5× improvement in compile rate.
2. A documented negative result showing that Knowledge Editing is unsuitable for neural decompilation, with analysis that decompilation errors stem from reasoning failures rather than knowledge gaps.
3. An error taxonomy categorizing decompilation failures into addressable patterns (45%) and fundamental limitations (55%), providing insight into the ceiling of achievable performance.
4. Evidence for a “syntactic-semantic gap” in neural decompilation, where syntactic correction is readily achievable but semantic reasoning remains challenging.
5. Demonstration that mixed training combining error-specific examples with general data achieves the highest functional correctness (28.08% Pass@1), representing an 81% relative improvement over baseline.

1.4 Thesis Outline

The remainder of this thesis is organized as follows.

Chapter 2: Background provides foundational material on decompilation, neural approaches to code transformation, parameter-efficient fine-tuning methods, and knowledge editing techniques.

Chapter 3: Related Work surveys existing research in neural decompilation and positions this work within the broader landscape.

Chapter 4: Methodology describes the experimental design, including the data pipeline, model configuration, training procedures, and evaluation framework.

Chapter 5: Results presents the experimental findings, including baseline performance, LoRA fine-tuning results, error analysis, and error-specific fine-tuning outcomes.

Chapter 6: Discussion interprets the results, examines implications for neural decompilation, and acknowledges limitations.

Chapter 7: Conclusion summarizes the findings and suggests directions for future work.

2 Background

This chapter introduces the foundational concepts required to understand the experimental work presented in this thesis. We begin with decompilation fundamentals, then cover the machine learning approaches under investigation: Parameter-Efficient Fine-Tuning and Knowledge Editing.

2.1 Decompilation Fundamentals

2.1.1 Compilation and Information Loss

Compilation transforms human-readable source code into machine-executable binary code through multiple stages: preprocessing, parsing, semantic analysis, optimization, and code generation. Each stage discards information that is unnecessary for execution but valuable for human understanding [1].

Variable names become register allocations or stack offsets. Type information is reduced to memory sizes and alignment constraints. Control flow structures like `for` and `while` loops compile to identical jump instructions. Comments and formatting disappear entirely. High-level abstractions such as struct layouts, inline functions, and macro expansions are flattened into sequences of machine instructions [1].

This information loss is inherently one-way. The compilation process is a many-to-one mapping: countless source programs compile to identical binaries. Recovering the original source from a binary is therefore fundamentally ill-posed; we can only approximate one of many possible originals.

2.1.2 Ghidra Decompiler

Modern decompilers like Ghidra [2], IDA Pro [7], and angr [8] employ sophisticated techniques including control flow graph reconstruction, data flow analysis, type recovery, and pattern matching for common idioms.

Ghidra’s decompiler follows a multi-stage pipeline that transforms binary machine code into C-like pseudocode. The process begins with **p-code generation**, where machine instructions are translated into Ghidra’s intermediate representation called p-code, a Register Transfer Language (RTL) designed specifically for reverse engineering. The SLEIGH specification language defines the translation from each processor’s machine code to p-code, enabling Ghidra to support multiple architectures through modular processor specifications [2].

From the raw p-code, Ghidra constructs **basic blocks and a control flow graph (CFG)**. Basic blocks are sequences of p-code operations with a single entry point and single exit point, connected by control flow edges representing jumps, branches, and function calls.

The CFG is normalized to ensure a unique entry block, which may require inserting placeholder blocks when the function's first instruction is a branch target [2].

The core analysis occurs in the **main simplification loop**, which iteratively refines the p-code representation. This loop first converts the p-code into Static Single Assignment (SSA) form, where each variable is assigned exactly once, enabling powerful dataflow optimizations. Dead code elimination removes operations whose results are never used, which is particularly important for decompilation since many machine instructions produce side effects (such as setting processor flags) that are irrelevant at particular program points. Type propagation infers high-level type information from instruction usage patterns and propagates this information through the SSA graph [2].

Term rewriting forms the bulk of the simplification process, applying transformation rules to normalize and simplify expressions. Unlike compiler optimizations that aim for performance, these rules target human readability: propagating copies, folding constants, simplifying algebraic expressions, and undoing compiler optimizations such as strength-reduced multiplications and divisions. The loop also recovers high-level control flow structures, identifying loops, if-else blocks, and switch statements from the CFG to enable structured output [2].

After simplification, Ghidra performs **variable recovery** by exiting SSA form and merging low-level variables into high-level variables. This process resembles register coloring in compilers, ensuring that merged variables do not hold conflicting values simultaneously. Additional merging passes reduce variable count further, attempting to produce code that resembles typical C programs. Finally, the decompiler selects variable names based on available symbol information or generates synthetic names, adds necessary type casts, and emits the final C tokens with syntax highlighting and address annotations for cross-referencing with the original binary [2].

Despite this sophisticated analysis, Ghidra's output differs substantially from original source code due to information loss during compilation. Ghidra produces pseudocode that is syntactically similar to C but contains characteristic artifacts of the recovery process [2].

Type annotations represent recovered data sizes rather than semantic types. Ghidra uses names like `undefined4` for 32-bit values, `undefined1` for bytes, and `undefined8` for 64-bit values when it cannot determine the original type. A variable declared as `int counter` in the source might appear as `undefined4 local_c` in the decompiled output [2].

Variable naming reflects stack layout rather than programmer intent. Local variables receive names like `local_10`, `local_1c`, or `local_28` based on their stack frame offsets. Parameters may appear as `param_1`, `param_2`, etc., losing the descriptive names from the original source [2].

Pointer and array handling often produces verbose expressions. Array accesses like `arr[i]` may decompile to pointer arithmetic such as `*(int *)((long)arr + (long)i * 4)`, with explicit casts reflecting the underlying memory operations [2].

String and data references use address-based names. String literals referenced via the GOT (Global Offset Table) may appear as `_LC0` or similar labels, while global data appears as `DAT_XXXXXXXX` with hexadecimal addresses [2].

Control flow reconstruction occasionally produces non-standard constructs. Compiler optimizations can create control flow patterns that do not map cleanly to standard C constructs, resulting in `goto` statements or awkward loop structures in the decompiled output [2].

Listing 1 illustrates these artifacts with a concrete example, showing how a simple function transforms through compilation and decompilation. The Ghidra output exhibits all the characteristic artifacts: type annotations (`undefined8`), synthesized variable names (`local_c`, `param_1`), explicit pointer arithmetic for array access, and non-standard return value handling via `CONCAT44`.

<pre>// Original source int sum_array(int *arr, int n) { int total = 0; for (int i = 0; i < n; i++) { total += arr[i]; } return total; }</pre>	<pre>// Ghidra pseudocode undefined8 sum_array(long param_1, int param_2) { undefined8 uVar1; int local_c; int local_8; local_c = 0; local_8 = 0; while (local_8 < param_2) { local_c = local_c + *(int *)(param_1 + (long)local_8 * 4); local_8 = local_8 + 1; } uVar1 = CONCAT44(local_c, local_c); return uVar1; }</pre>
---	--

Listing 1: Original C source code (left) and Ghidra decompiled pseudocode (right).

While this output aids human reverse engineers in understanding program behavior, it typically cannot be directly compiled due to the non-standard type annotations and constructs. The gap between decompiler output and compilable C source code motivates the neural decompilation approach explored in this thesis.

2.2 Transformer Architecture

The Transformer architecture [9] forms the foundation of modern Large Language Models. Unlike earlier recurrent architectures [10] that process sequences token-by-token, Transformers process entire sequences in parallel through a mechanism called self-attention. This section introduces the key components relevant to understanding the adaptation methods investigated in this thesis.

Before processing, input text must be converted into discrete tokens through tokenization. A token is the fundamental unit of text that the model processes; neural networks operate

on numerical vectors, so text must be segmented into units that can be mapped to vector representations. Modern LLMs use subword tokenization algorithms such as Byte-Pair Encoding (BPE) [11], which iteratively merge frequent character pairs to build a vocabulary of common subunits. This allows common words to become single tokens while rare words are decomposed (e.g., "decompilation" $\rightarrow$ ["de", "compil", "ation"]), balancing vocabulary size against sequence length. These tokens are then mapped to dense vector embeddings that serve as input to the Transformer layers [9].

2.2.1 Self-Attention Mechanism

Self-attention allows each token in a sequence to attend to all other tokens, learning contextual relationships regardless of their distance in a sequence. For an input sequence of token embeddings $X \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the embedding dimension, self-attention computes three matrices through learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are the query, key, and value projection matrices respectively [9]. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large, which would push the softmax into regions with extremely small gradients [9].

In practice, Transformers employ multi-head attention, which runs several attention operations in parallel with different learned projections:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

where each $\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$ and W_O is the output projection matrix. This allows the model to attend to information from different representation subspaces at different positions [9].

The projection matrices (W_Q, W_K, W_V, W_O) are key targets for parameter-efficient fine-tuning methods like LoRA [4], as they control how the model forms and combines attention patterns.

2.2.2 Feed-Forward Layers

Each Transformer layer contains a position-wise feed-forward network (FFN) applied independently to each token representation after the attention sublayer. In the original Transformer architecture [9], the FFN consists of two linear transformations with a ReLU activation:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

where $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ projects from the model dimension d to a larger intermediate dimension d_{ff} , and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ projects back to the model dimension.

Modern architectures like Llama [12] employ a gated variant using the SwiGLU activation [13]:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_{\text{gate}}) \odot xW_{\text{up}})W_{\text{down}}$$

where $\text{SiLU}(x) = x \odot \sigma(x)$ is the Sigmoid Linear Unit activation, $\odot$ denotes element-wise multiplication. The weight matrices $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ project to the intermediate dimension while $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$ projects back. This gated formulation improves training stability and model quality [13].

The feed-forward layers serve as the primary site for storing factual knowledge in Transformer models [5]. While attention layers route information between token positions, FFN layers transform individual token representations, acting as key-value memories [14]. This property makes FFN layers the target of Knowledge Editing methods like ROME [5], and explains why LoRA fine-tuning [4] targets both attention projections and FFN projections to enable comprehensive adaptation.

2.2.3 Layer Composition

A complete Transformer layer combines the attention and feed-forward sublayers with residual connections [15] and normalization. The original Transformer applies “post-norm”, placing normalization after each sublayer [9]. Modern architectures like Llama [12] instead use “pre-norm”, applying normalization before each sublayer. Llama replaces standard layer normalization [16] with RMSNorm [17], which omits the mean-centering step while achieving comparable performance with reduced computational overhead.

The pre-norm formulation is:

$$\begin{aligned} h &= x + \text{MultiHead}(\text{RMSNorm}(x)) \\ y &= h + \text{FFN}(\text{RMSNorm}(h)) \end{aligned}$$

Here x denotes the input to the layer, h the intermediate representation after attention, and y the final layer output.

Beyond normalization, the residual connections [15] are critical architectural components that enable gradient flow through deep networks and allow each layer to learn incremental refinements rather than complete transformations. Modern LLMs stack dozens of these layers (CodeLlama-7B uses 32 layers [3]) creating a deep processing pipeline where each layer refines the representations produced by previous layers.

This compositional structure has implications for adaptation methods investigated in this thesis. LoRA [4] targets the projection matrices within both attention and FFN sublayers, with the residual connections ensuring that adaptations combine additively with the frozen base model computations. This layer-wise organization also enables Knowledge Editing methods like ROME to localize factual knowledge to specific middle layers, where causal

tracing experiments have shown factual associations are most strongly encoded in the MLP sublayers [5].

2.3 Large Language Models

Large Language Models (LLMs) are Transformer-based models trained on massive text corpora, typically containing hundreds of billions to trillions of tokens. The scaling of both model parameters and training data has led to emergent capabilities not present in smaller models [18].

2.3.1 Pretraining and Scale

LLMs are pretrained using **next-token prediction**, also called causal language modeling. Given a sequence of tokens $x_1, x_2, \dots, x_{\{t-1\}}$, the model learns to predict the probability distribution over the next token x_t [19]:

$$\mathcal{L} = - \sum_{t=1}^T \log P(x_t \mid x_1, \dots, x_{\{t-1\}})$$

This self-supervised objective requires no manual labeling; the training signal comes from the text itself. The model learns syntactic patterns, semantic relationships, and factual knowledge implicitly through the prediction task [19].

Scaling laws [20] describe predictable relationships between model size, dataset size, compute budget, and model performance. These empirical findings show that performance improves smoothly as a power law with increased scale, guiding decisions about resource allocation during training.

The Llama family of models [21] exemplifies efficient scaling. Llama 2 [12] ranges from 7B to 70B parameters, trained on 2 trillion tokens of publicly available text. Architectural choices including RMSNorm [17], SwiGLU activations [13], and rotary positional embedding (RoPE) [22] improve training stability and model quality compared to the original Transformer design.

2.3.2 LLMs for Code

Code-specialized LLMs adapt the language model paradigm to programming languages. While general-purpose LLMs encounter code during pretraining, dedicated code models are trained predominantly on source code, learning programming-specific patterns such as syntax rules, API usage, and algorithmic idioms [3].

CodeLlama [3] extends Llama 2 through continued pretraining on 500 billion tokens of code-heavy data. The training mixture emphasizes programming languages (primarily Python, C/C++, Java, and JavaScript) while retaining natural language capability for understanding comments and documentation. This specialization produces models that outperform general-purpose LLMs of equivalent size on code generation benchmarks [3].

CodeLlama includes several variants optimized for different use cases. The base model excels at code completion, while CodeLlama-Instruct undergoes additional fine-tuning to follow natural language instructions. Instruction tuning [23] teaches the model to respond appropriately to user requests rather than simply continuing text, making it suitable for task-oriented applications like code generation from specifications.

The instruction-tuned variant is particularly relevant for decompilation, where the model must follow explicit directives to transform pseudocode into valid C code. The 7B parameter size provides a practical balance: sufficient capacity for complex code transformations while remaining tractable for parameter-efficient fine-tuning on consumer hardware.

2.4 Neural Decompilation

Neural decompilation frames the refinement of decompiler output as a sequence-to-sequence translation task [24]. Rather than reconstructing source code from raw binaries, neural approaches take existing decompiler output as input and generate improved, compilable code as output.

This framing has several advantages. First, traditional decompilers handle the complex low-level analysis; recovering control flow, identifying function boundaries, and inferring approximate types. The neural model then focuses on the higher-level task of producing natural, compilable code. Second, the approach can leverage the large body of work on neural machine translation and code generation.

Large Language Models pretrained on code, such as CodeLlama [3], have shown strong performance on code generation tasks. Fine-tuning these models on decompilation data allows them to learn the mapping from decompiler artifacts to conventional C idioms. The model learns to replace undefined4 with appropriate types, generate meaningful variable names, and restructure awkward control flow into idiomatic patterns.

2.5 Parameter-Efficient Fine-Tuning

Several parameter-efficient fine-tuning methods have been proposed including adapter modules [25], prefix tuning [26], and prompt tuning [27]. This thesis employs Low-Rank Adaptation (LoRA) [4] because it introduces no inference latency (adapters can be merged into base weights), is compatible with quantization (enabling QLoRA [28]), and provides fine-grained control over which layers to adapt.

2.5.1 The Challenge of Full Fine-Tuning

Fine-tuning adapts a pretrained model to a specific task by updating its parameters on task-specific data. For modern Large Language Models with billions of parameters, full fine-tuning presents significant challenges. Updating all parameters requires storing optimizer states and gradients for each parameter, often exceeding available GPU memory.

Furthermore, each fine-tuned model requires storing a complete copy of all parameters, making it expensive to maintain multiple specialized models.

2.5.2 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) [4] addresses these challenges by freezing the pretrained model weights and injecting trainable low-rank decomposition matrices into each layer. The approach is motivated by evidence that weight updates during fine-tuning have low “intrinsic rank”; the adaptation can be effectively captured in a much lower-dimensional subspace than the full parameter space [4].

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA adds a parallel path:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} \cdot B A x$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ are small trainable matrices with rank $r \ll \min(d, k)$, and α is a scaling hyperparameter that controls the magnitude of the adaptation. During training, only A and B are updated while W_0 remains frozen [4].

A key design choice is the initialization: A is initialized from a random Gaussian distribution while B is initialized to zero. This ensures $\Delta W = BA = 0$ at the start of training, so the model begins from its exact pretrained state and learns adaptations incrementally [4].

This approach dramatically reduces trainable parameters. For a model with $d = 4096$, applying LoRA with rank $r = 64$ to a weight matrix reduces trainable parameters from 16 million to approximately 500 thousand; a $32\times$ reduction.

2.5.3 QLoRA

QLoRA [28] extends LoRA by combining it with quantization, enabling fine-tuning of models that would otherwise exceed available memory. The frozen base model weights are stored in 4-bit precision using a novel NormalFloat (NF4) data type. NF4 is specifically optimized for neural network weights, which are approximately normally distributed after pretraining; its quantization bins are spaced according to quantiles of the normal distribution, minimizing information loss compared to uniform quantization schemes. The low-rank adapters remain in higher precision (typically 16-bit) for training stability, and gradients are computed through the quantized weights via dequantization during the forward pass [28].

This combination reduces memory substantially. A 7-billion parameter model that requires approximately 28GB in 16-bit precision can be loaded in roughly 4GB with 4-bit quantization, making fine-tuning feasible on consumer hardware [28].

2.6 Knowledge Editing

Knowledge Editing encompasses various techniques for targeted model modification, including hypernetwork-based approaches [29], semi-parametric methods with explicit memory [30], and in-context editing [31]. Unlike fine-tuning, which updates parameters across the entire model, Knowledge Editing aims to surgically modify specific factual associations or behaviors.

This thesis investigates Rank-One Model Editing (ROME) [5], which localizes factual knowledge to specific MLP layers in Transformer models [9] and modifies their weights to update individual facts. The technique treats certain MLP layers as key-value stores [14] and uses rank-one updates to change specific associations while preserving other model capabilities. ROME was selected due to its interpretable mechanism, theoretical grounding through causal tracing, and efficient single-layer modification.

ROME and related methods were originally developed for correcting factual errors in language models; for example, updating a model’s knowledge that “The Eiffel Tower is located in Paris” to “The Eiffel Tower is located in London”. This thesis investigates whether Knowledge Editing can similarly correct specific decompilation errors; treating incorrect code patterns as “facts” to be edited.

2.7 Evaluation Metrics

Evaluating decompilation quality requires metrics that capture functional correctness rather than surface-level textual similarity. A decompiled function may use different variable names, formatting, or even algorithmic approaches while remaining functionally equivalent to the original.

Pass@ k [32] measures the probability that at least one of k generated samples passes all test cases. For Pass@1, each generated function is compiled and executed against a test harness containing assertions that verify functional behavior. A function passes only if it compiles successfully, executes without errors, and produces correct outputs for all test inputs.

Compile rate measures the fraction of generated functions that successfully compile, independent of functional correctness. The metric captures the model’s ability to produce syntactically valid code; a prerequisite for any practical use.

These metrics are preferable to text-based metrics like BLEU [33] or exact match, which can penalize functionally correct code that differs stylistically from reference implementations. A function that correctly implements a specification with different variable names receives full credit under Pass@ k but may score poorly on textual similarity metrics.

3 Related Work

This chapter situates the present work within the broader landscape of neural decompilation research and highlights the specific gap this thesis addresses.

3.1 Neural Decompilation Systems

Early approaches to neural decompilation employed recurrent neural networks to translate assembly or low-level intermediate representations directly to source code. Coda [24] pioneered the end-to-end neural decompiler concept, training sequence-to-sequence models to transform binary representations into C code.

More recent work has leveraged the capabilities of Large Language Models. LLM4Decompile [34] represents the current state-of-the-art, training a series of models ranging from 1.3B to 33B parameters specifically for decompilation. Their approach follows two paradigms: end-to-end decompilation from assembly, and refinement of existing decompiler output. The refinement approach, which takes Ghidra pseudocode as input, achieved substantially higher re-executability rates than the end-to-end method.

Other LLM-based approaches include DecGPT [35], which refines IDA Pro output by incorporating compiler error messages into an iterative refinement loop, and DeGPT [36], which focuses on improving the readability of Ghidra output for human analysts. Nova [37] developed specialized binary language models fine-tuned for decompilation tasks.

These systems share a common methodology: they either train or fine-tune models on large corpora of binary-source pairs, updating all model parameters. LLM4Decompile, for instance, trains on billions of tokens of decompilation data. This full fine-tuning approach achieves strong results but requires substantial computational resources and produces task-specific models that cannot easily incorporate targeted corrections.

3.2 Research Gap and Positioning

While existing work has demonstrated the effectiveness of LLMs for decompilation, the field has not systematically explored parameter-efficient adaptation methods. All major neural decompilation systems employ either full fine-tuning or prompting-based approaches.

This thesis investigates whether parameter-efficient techniques can achieve competitive results while enabling more flexible model adaptation. Specifically, we compare:

1. **Low-Rank Adaptation (LoRA)** [4] as a parameter-efficient alternative to full fine-tuning, examining whether the dramatic reduction in trainable parameters compromises decompilation quality.
2. **Knowledge Editing** [5] as a surgical intervention technique, testing whether decompilation errors can be corrected through targeted weight modifications rather than broad training.

3. **Error-specific LoRA training** on curated datasets, combining the efficiency of LoRA with focused training on particular error categories.

This comparison addresses a practical concern: as LLMs grow larger, full fine-tuning becomes increasingly prohibitive. If parameter-efficient methods can match or approach full fine-tuning performance, they offer a more accessible path to specialized decompilation models.

Furthermore, only a few prior works have investigated Knowledge Editing for code transformation tasks [38]. While ROME [5] and related techniques have shown success in correcting factual knowledge, their applicability to structural code transformations remains unexplored. This thesis provides the first empirical evaluation of Knowledge Editing in the decompilation domain.

4 Methodology

4.1 Overview

This thesis investigates two fundamentally different paradigms for adapting pre-trained Large Language Models to the task of neural decompilation: Low-Rank Adaptation (LoRA) [4] and Knowledge Editing (KE) [5]. These approaches represent contrasting philosophies in model adaptation and are hypothesized to offer complementary strengths when addressing the challenges inherent to decompilation.

The decompilation task is formulated as a translation problem where the input consists of Ghidra-generated pseudocode rather than raw binary or assembly. This intermediate representation retains essential low-level semantics while providing a more structured input format that is better suited for LLM processing. The objective is to transform this pseudocode into clean, idiomatic, and functionally equivalent high-level C code.

Low-Rank Adaptation constitutes a global adaptation strategy. By introducing low-rank trainable matrices into the Transformer architecture, LoRA enables the model to learn broad patterns from a corpus of decompilation examples [4]. This approach is well-suited for capturing general improvements such as more idiomatic code generation, better recognition of common programming constructs, and improved handling of compiler-introduced patterns. This adaptation affects the model’s behavior across a wide range of inputs, making it appropriate for enhancing overall decompilation quality.

Knowledge Editing, in contrast, represents a surgical intervention approach. However preliminary investigation revealed that traditional KE techniques such as ROME [5], designed primarily for factual knowledge correction in natural language domains, are not well-suited to the decompilation task, which involves complex structural transformations rather than discrete factual assertions. Consequently, this work explores **error-specific LoRAs** as an alternative targeted adaptation strategy. Rather than editing model weights directly, error-specific LoRAs are trained on curated datasets focusing on particular recurring error patterns (e.g., loop bound errors, operator errors, or initialization errors). This approach maintains the surgical, targeted philosophy of knowledge editing while remaining compatible with the architectural and task characteristics of neural decompilation.

The central hypothesis of this work proposes that these two approaches, general-purpose LoRAs and error-specific LoRAs, are not merely alternatives but rather address different aspects of the decompilation problem. General LoRA is expected to yield consistent improvements in overall code quality and readability, while error-specific LoRAs are hypothesized to excel at eliminating particular, reproducible error categories that persist even after broad fine-tuning. Furthermore, a hybrid approach combining both techniques may leverage their respective strengths to achieve superior results compared to either method in isolation.

The experimental design follows a structured comparative methodology. First, a baseline is established by evaluating a pre-trained LLM on a curated dataset of binary functions with known source code. Subsequently, general LoRA fine-tuning is applied to create a globally adapted model, and error-specific LoRAs are trained and applied to create targeted-correction variants. All adapted models are evaluated against the baseline using identical metrics (compilability, and functional equivalence), enabling direct comparison.

The complete pipeline is illustrated in Figure 1: C source code is compiled with GCC at -O2 optimization, decompiled by Ghidra to pseudocode, refined by the LLM, and evaluated through compilation and functional testing.



Figure 1: Data pipeline overview.

4.2 Data Pipeline

The experimental infrastructure relies on two complementary datasets serving distinct purposes in the evaluation pipeline. **ExeBench** [39] serves as the primary training corpus, providing a diverse collection of C functions suitable for fine-tuning the models.

HumanEval-Decompile [34] functions as the evaluation dataset, offering 151 programming problems with associated test harnesses that enable rigorous verification of functional equivalence beyond mere syntactic similarity.

The data preparation process begins with **compilation**, where source C code is compiled into binary executables using GCC [40] with the -O2 optimization level. This optimization setting represents a realistic balance between performance and debuggability commonly used in production software, producing binaries with substantial compiler transformations, including loop optimizations, function inlining, and register allocation, while avoiding the most aggressive optimizations that can make decompilation exceptionally challenging.

Following compilation, **decompilation** is performed using Ghidra in headless mode, enabling automated batch processing of binaries. Ghidra’s decompiler analyzes each compiled binary and generates pseudocode that serves as the input representation for the neural models. This pseudocode retains low-level semantics such as explicit type casts, pointer arithmetic, and architecture-specific idioms while providing more structure than raw assembly.

To ensure dataset quality, a **filtering and quality assurance** process is applied to the generated pairs of Ghidra pseudocode and original source code. Filtering criteria include line length constraints to remove excessively long or short functions that may represent edge cases, length ratio checks between pseudocode and source code to identify potential decompilation failures or anomalies, and validation to ensure both input and output constitute valid, complete function pairs. These filters remove malformed samples that could introduce noise during training or evaluation.

The resulting dataset comprises approximately **4,000 training samples** from ExeBench and **151 test samples** from HumanEval-Decompile, each accompanied by test harnesses that enable automated verification of functional correctness through execution-based testing.

4.3 LoRA Fine-Tuning Approach

4.3.1 Base Model Selection

The foundation for LoRA fine-tuning is CodeLlama-7B-Instruct, a 7-billion parameter large language model specifically optimized for code-related tasks. CodeLlama [3] represents a family of models derived from Llama 2 [12], further trained on code-heavy corpora to develop stronger capabilities in code understanding, generation, and transformation tasks. The Instruct variant has been additionally fine-tuned to follow instructions, making it particularly well-suited for task-oriented applications where the model must respond to structured prompts.

Several factors motivated this selection. First, the 7B parameter scale strikes a practical balance between model capacity and computational feasibility, enabling relatively efficient fine-tuning and inference while maintaining sufficient representational power for complex code transformations. Second, CodeLlama’s architecture and training specifically target programming language understanding, providing a stronger starting point than general-purpose language models. Third, the Instruct variant’s instruction-following capabilities align well with the decompilation task formulation, where the model receives explicit directives to transform Ghidra pseudocode into clean C code. Finally the model’s open availability and extensive community adoption provide valuable resources for implementation and comparison.

4.3.2 Training Configuration

The training infrastructure employs **Unsloth** [41], an optimized framework for efficient LLM fine-tuning that provides significant speedups over standard implementations. The base model is loaded in 4-bit precision [28], reducing memory requirements while maintaining model quality. Unsloth’s custom gradient checkpointing implementation further reduces memory overhead compared to standard PyTorch [42] gradient checkpointing.

The LoRA configuration targets all linear projection layers within the Transformer architecture. For the attention mechanism, this includes the query, key, value, and output projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`). For the feed-forward network, the gated MLP projections are targeted (`gate_proj`, `up_proj`, `down_proj`). This comprehensive targeting ensures that adaptations can occur throughout the model’s representational pipeline. Unlike some configurations that apply dropout to LoRA layers, dropout is set to zero for improved training stability with small datasets.

The training hyperparameters are configured as follows. A per-device batch size of 2 is used with gradient accumulation over 8 steps, yielding an effective batch size of 16. The

maximum sequence length is set to 2048 tokens, sufficient to accommodate most function pairs in the dataset. Training proceeds for 3 epochs with a learning rate of 2×10^{-4} and a linear learning rate scheduler with a warmup ratio of 0.05. The optimizer is AdamW [43] with 8-bit states. A weight decay of 0.01 provides light regularization. A fixed random seed of 3407 ensures reproducibility across training runs.

Training was conducted using cloud and high-performance computing resources. The majority of LoRA training runs utilized Google Colab [44] instances with a single NVIDIA A100 GPU (40GB VRAM). Selected experiments were conducted on the bwHPC cluster [45] provided by the state of Baden-Württemberg, using nodes with four NVIDIA H100 GPUs.

Parameter	Value
Framework	Unsloth
Quantization	4-bit
LoRA targets	All attention + MLP projections
LoRA dropout	0
Effective batch size	16 (2×8 accumulation)
Max sequence length	2048 tokens
Epochs	3
Learning rate	2×10^{-4}
LR scheduler	Linear
Optimizer	AdamW 8-bit
Weight decay	0.01
Warmup ratio	0.05
Random seed	3407

Table 1: LoRA training configuration using Unsloth

Prompt template design plays a crucial role in eliciting appropriate model behavior. The prompt structure leverages CodeLlama’s instruction format, using the [INST] tags that the model was trained to recognize. The template is designed to address common failure modes observed during preliminary experiments:

```
<s>[INST] You are an expert C decompiler.
Refine the following Ghidra pseudocode into valid, compilable C code.
STRICT RESPONSE RULES:
1. Do not write a main function.
```

2. Keep the exact same function name and arguments.
3. Output ONLY the raw code. Do not use Markdown code blocks (```).
4. Do not output any introductory text or explanations.

Pseudocode:

```
{ghidra_pseudocode}
[/INST]
```

4.3.3 Hyperparameter Search

To identify optimal LoRA hyperparameters, a grid search was conducted over rank values ranging from 8 to 128, with alpha values set equal to or double the rank. Each configuration was evaluated across 5 independent runs on the HumanEval-Decompile test set (n=151) to account for sampling variability.

Config	Rank / Alpha	Pass@1 (%)	Compile (%)
Baseline	—	15.50 ± 1.08	18.94 ± 0.53
LoRA	8 / 8	20.93 ± 0.79	82.38 ± 0.32
LoRA	16 / 16	21.46 ± 2.03	82.91 ± 1.59
LoRA	32 / 32	23.44 ± 1.60	84.77 ± 0.59
LoRA	32 / 64	23.71 ± 1.84	83.97 ± 0.77
LoRA	64 / 64	23.95 ± 1.35	84.33 ± 1.19
LoRA	64 / 128	22.78 ± 0.90	85.56 ± 1.59
LoRA	128 / 128	23.18 ± 2.09	84.50 ± 0.99

Table 2: LoRA hyperparameter search results on HumanEval-Decompile (5 runs per config). Bold indicates best performance.

The results reveal several patterns. First, even the smallest LoRA configuration (r=8) dramatically improves compilability from 18.94% to 82.38%, indicating that fine-tuning effectively teaches the model to generate syntactically valid C code rather than pseudocode artifacts. Second, Pass@1 shows diminishing returns beyond rank 32, with configurations from r=32 to r=128 all achieving approximately 23-24%. Third, increasing alpha relative to rank (e.g., r=32/a=64 vs r=32/a=32) provides marginal benefit. Based on these results, the r=64/a=64 configuration was selected for subsequent experiments as it achieved the highest mean Pass@1 while maintaining stable performance across runs.

4.4 Knowledge Editing Approach

Knowledge Editing (KE) represents a fundamentally different paradigm from fine-tuning: rather than updating model weights through gradient descent over training examples, KE methods directly modify specific parameters to alter targeted factual associations while preserving other model behaviors. This section describes the experimental investigation of ROME (Rank-One Model Editing) [5] for neural decompilation, conducted to evaluate whether surgical weight modifications can address systematic decompilation errors.

4.4.1 Formulating Decompilation as Knowledge Editing

The hypothesis underlying this experiment was that certain Ghidra pseudocode artifacts could be conceptualized as incorrect factual associations that Knowledge Editing might correct. Ghidra’s decompiler produces type annotations such as `undefined4`, `undefined1`, and `undefined8` which correspond to C types `int`, `char`, and `long` respectively. If the base LLM fails to consistently translate these artifacts, this failure might stem from incorrect or weak factual associations that ROME could strengthen.

Under this formulation, the decompilation task contains implicit knowledge triplets of the form (subject, relation, object):

- (`undefined4`, “corresponds to C type”, `int`)
- (`undefined1`, “corresponds to C type”, `char`)
- (`undefined8`, “corresponds to C type”, `long`)

Additionally, Ghidra produces context-specific artifacts such as `_LC0` for string literal references and `DAT_XXXXXXX` for data section addresses, though these lack fixed target values and require context-dependent resolution.

4.4.2 Implementation

ROME [5] edits were implemented using the EasyEdit library [46], which provides a standardized interface for various Knowledge Editing methods. Edit requests were specified as JSON objects containing the subject (Ghidra artifact), relation (type correspondence), and target object (C type). The editing process modifies a single feed-forward layer in the Transformer, identified through causal tracing as the layer where factual associations are stored [5].

Four edit requests were created targeting the most common Ghidra type artifacts. The edited model was then tested on the same decompilation prompts used for baseline evaluation, and the generated outputs were analyzed for artifact removal rates.

4.4.3 Fundamental Limitations

The experiment revealed a fundamental mismatch between Knowledge Editing and the decompilation task. Critically, the base CodeLlama model already possesses the relevant factual knowledge; when directly asked “What C type does `undefined4` represent?”, the model correctly responds “`int`”. The decompilation failures therefore do not stem from

missing factual associations but from inconsistent application of known facts during code generation. In testing, none of the four edit requests produced correct factual responses, and artifact removal improved only marginally, with the one removed artifact replaced by an incorrect value.

This distinction is crucial: Knowledge Editing methods are designed to update atomic factual associations (e.g., changing “The Eiffel Tower is located in Paris” to “The Eiffel Tower is located in London”). Neural decompilation, however, requires consistent application of transformation rules across diverse syntactic contexts, context-dependent reasoning about surrounding code, and multi-step inference to reconstruct program semantics. These capabilities fall outside the scope of what Knowledge Editing can address.

Furthermore, many Ghidra artifacts cannot be formulated as fixed factual mappings. String literal references like `_LC0` must be resolved based on the surrounding code context; there is no single correct replacement that could be encoded as an edited fact. The model must reason about each occurrence individually.

Based on these findings, Knowledge Editing was determined to be unsuitable for the neural decompilation task. The remainder of this thesis therefore focuses on LoRA fine-tuning as the primary adaptation method, with Knowledge Editing serving as a documented negative result that clarifies the nature of the decompilation challenge.

4.5 Error Analysis

To understand the limitations of LoRA fine-tuning and identify opportunities for targeted improvement, a systematic error analysis was conducted on the evaluation results. This analysis aimed to categorize failure modes and assess which error patterns might be addressable through additional training interventions.

4.5.1 Evaluation Framework

The evaluation framework assesses generated code through a three-stage pipeline. First, **compilation checking** verifies that the generated C code is syntactically valid by attempting compilation with GCC. Second, **functional equivalence testing** executes the compiled code against a test harness containing assertions that verify input-output behavior matches the ground truth implementation. Third, **stability assessment** runs each evaluation multiple times (5 runs per configuration) to distinguish consistent failures from stochastic variations in generation.

This multi-run approach revealed three categories of samples: consistently passing (same result across all runs), consistently failing, and flaky samples that pass in some runs but fail in others. The flaky category is particularly informative, as it indicates samples where the model’s output is near the decision boundary; sometimes producing correct code and sometimes not.

4.5.2 Failure Categorization

Consistent failures were further categorized by failure type: compilation errors (syntactically invalid code), assertion failures (compiles but produces incorrect output), and timeouts (execution exceeds time limit). Assertion failures are of particular interest as they represent cases where the model generates plausible but semantically incorrect code; a more subtle failure mode than outright syntax errors.

4.5.3 Semantic Error Taxonomy

Manual analysis of assertion failures revealed six distinct error patterns:

- **Decompiler information loss:** Ghidra produces severely degraded output (e.g., stub functions) that lacks essential logic. These failures are unrecoverable regardless of model capability.
- **Algorithm re-interpretation:** The model generates a structurally different algorithm that appears plausible given the ambiguous pseudocode but produces incorrect results.
- **Loop bound and index errors:** Incorrect loop termination conditions or array indexing, often manifesting as off-by-one errors (e.g., $i < n$ vs. $i \leq n$).
- **Comparison operator errors:** Incorrect relational operators that change program semantics (e.g., $<$ vs. $>$ in sorting comparisons).
- **Variable initialization errors:** Missing initialization or incorrect use of output parameters, such as dereferencing pointers before assignment.
- **String and format handling errors:** Incorrect format specifiers or string operations that produce malformed output.

4.5.4 Trainability Assessment

Each error pattern was assessed for potential addressability through synthetic training data. Patterns involving discrete, systematic transformations (such as loop bound variations and operator substitutions) were identified as potentially trainable. Patterns requiring semantic reasoning or suffering from fundamental information loss were classified as likely unaddressable through fine-tuning alone.

This taxonomy informed the design of error-specific training data, where synthetic examples were generated by deliberately introducing the identified error patterns into correct code, training the model to recognize and correct these specific failure modes.

4.6 Comparison Strategy

The original research design proposed comparing two adaptation approaches: LoRA for general fine-tuning and Knowledge Editing for targeted corrections. However, as documented in the previous section, Knowledge Editing proved fundamentally unsuitable for the decompilation task; the model already possesses the relevant factual knowledge but

fails to apply it consistently during generation. This finding itself constitutes a contribution, clarifying that neural decompilation errors stem from reasoning failures rather than knowledge gaps. Consequently, the experimental evaluation focuses on assessing LoRA’s effectiveness against the unadapted baseline, with the Knowledge Editing investigation serving as a documented negative result that illuminates the nature of the decompilation challenge.

5 Results

This chapter presents the experimental findings from evaluating LoRA-based fine-tuning for neural decompilation. All evaluations use the HumanEval-Decompile test set (n=151) with generated code compiled via GCC (-O2 optimization) and executed against test harnesses with a 2-second timeout. Code generation employs nucleus sampling (temperature 0.2, top-p 0.95) [47], and each configuration is evaluated across five independent runs to account for sampling variability.

5.1 Baseline Performance

The baseline establishes the capabilities of the pre-trained CodeLlama-7B-Instruct model applied to neural decompilation without task-specific adaptation.

Metric	Mean	Std Dev
Pass@1 (%)	15.50%	$\pm 1.08\%$
Compile (%)	18.94%	$\pm 0.53\%$

Table 3: Baseline model performance on HumanEval-Decompile (n=151, 5 runs)

The baseline achieves a Pass@1 of 15.50% and a compile rate of only 18.94%. The notably low compile rate reveals that the untuned model frequently reproduces Ghidra-specific artifacts, such as undefined4 type annotations and non-standard syntax, rather than generating valid C code. This indicates that the primary challenge is not semantic reasoning alone but also syntactic adaptation: the model must learn to translate decompiler idioms into standard C constructs.

5.2 LoRA Fine-Tuning Results

LoRA fine-tuning produces substantial improvements over the baseline. The hyperparameter search results, detailed in Table 2, identified the r=64, a=64 configuration as optimal, achieving 23.95% Pass@1.

Model	Pass@1 (%)	Compile (%)	Δ Pass@1
Baseline	15.50 $\pm$ 1.08	18.94 $\pm$ 0.53	—
LoRA (r=64, a=64)	23.95 $\pm$ 1.35	84.33 $\pm$ 1.19	+8.45

Table 4: Comparison of baseline and best LoRA configuration

The most striking result is the compile rate improvement: from 18.94% to 84.33%, a 4.5 $\times$ increase. This demonstrates that LoRA effectively teaches the model to generate

syntactically valid C code, eliminating most Ghidra artifacts that caused compilation failures.

The Pass@1 improvement from 15.50% to 23.95% represents a 54% relative gain, though the absolute improvement of 8.45 percentage points is more modest. This asymmetry between compile rate and functional correctness improvements suggests that LoRA excels at syntactic correction, while consistent semantic reasoning remains difficult to achieve.

Listing 2 illustrates a case where LoRA succeeds at both syntactic and semantic correction. The Ghidra input shows compiler loop unrolling artifacts that obscure a simple increment-each-element operation. The baseline model reproduces this convoluted structure and produces semantically incorrect code, while the LoRA fine-tuned model recovers the original loop structure and generates correct, idiomatic C.

Ghidra Input:

```
undefined8 func0(int *param_1,uint param_2)
{
    int *piVar1;

    if (0 < (int)param_2) {
        piVar1 = param_1 + (int)param_2;
        if ((param_2 & 1) != 0) {
            *param_1 = *param_1 + 1;
            param_1 = param_1 + 1;
            if (param_1 == piVar1) {
                return 0;
            }
        }
        do {
            *param_1 = *param_1 + 1;
            param_1[1] = param_1[1] + 1;
            param_1 = param_1 + 2;
        } while (param_1 != piVar1);
    }
    return 0;
}
```

Baseline Output (assertion failure):

```
int func0(int *param_1, uint param_2)
{
    int *piVar1 = param_1 + (int)param_2;
    if (param_2 & 1) {
        *param_1 = *param_1 + 1;
        param_1 = param_1 + 1;
        if (param_1 == piVar1)
            return 0;
    }
    do {
        *param_1 = *param_1 + 1;
        param_1[1] = param_1[1] + 1;
        param_1 = param_1 + 2;
    } while (param_1 != piVar1);
    return 0;
}
```

LoRA Output (passes):

```
void func0(int *p, int n)
{
    int i;
    for (i = 0; i < n; i++)
        *p++ += 1;
}
```

Listing 2: Qualitative comparison of model outputs on a function with loop unrolling artifacts.

5.2.1 Result Stability

Evaluation across multiple runs reveals variability in model performance due to sampling randomness. Analysis of per-sample consistency across five evaluation runs identified three categories:

Category	Count	Percentage
Consistently passing	20	13.2%
Consistently failing	99	65.6%
Flaky (inconsistent)	32	21.2%

Table 5: Result consistency across 5 evaluation runs (n=151)

The 21.2% of samples exhibiting flaky behavior, sometimes passing, sometimes failing, indicates that for a subset of problems, the model operates near the decision boundary. Minor variations in token sampling can lead to functionally different outputs. This underscores the importance of multi-run evaluation: single-run metrics would misrepresent true model capability.

5.3 Error Analysis

To understand the limitations of LoRA fine-tuning, the 99 consistently failing samples were analyzed.

Failure Type	Count	Percentage
Assertion failure	76	76.8%
Compilation error	22	22.2%
Timeout	1	1.0%

Table 6: Failure type distribution among consistently failing samples

The dominance of assertion failures (76.8%) over compilation errors indicates that the fine-tuned model successfully generates compilable code in most cases, but the generated code does not always preserve the original program’s semantics.

5.3.1 Semantic Error Patterns

Manual analysis of the assertion failures revealed six distinct error categories:

Decompiler information loss (~10% of failures): Ghidra produces severely degraded output lacking essential logic. When the input contains only a stub function like `undefined8 func0(void) { return 0; }`, no model can reconstruct the original semantics. These represent data quality limitations rather than model failures.

Algorithm re-interpretation (~35% of failures): The model generates a semantically different but plausible algorithm (e.g., an array interspersing task implemented with modulo-based indexing instead of the ground truth’s dual-iterator approach). Both implementations appear reasonable given the ambiguous pseudocode, but only one matches the test harness expectations.

Loop bound errors (~20% of failures): Incorrect termination conditions, often manifesting as off-by-one errors. Common patterns include $i < n$ versus $i \leq n$ or $i < n - 1$ versus $i < n$.

Comparison operator errors (~15% of failures): Incorrect relational operators that alter program semantics, such as using $>$ instead of $<$ in sorting comparisons.

Variable initialization errors (~10% of failures): Missing initialization of variables, particularly output parameters used before being set.

String and format handling errors (~10% of failures): Incorrect format specifiers or string operations, such as missing space separators in formatted output.

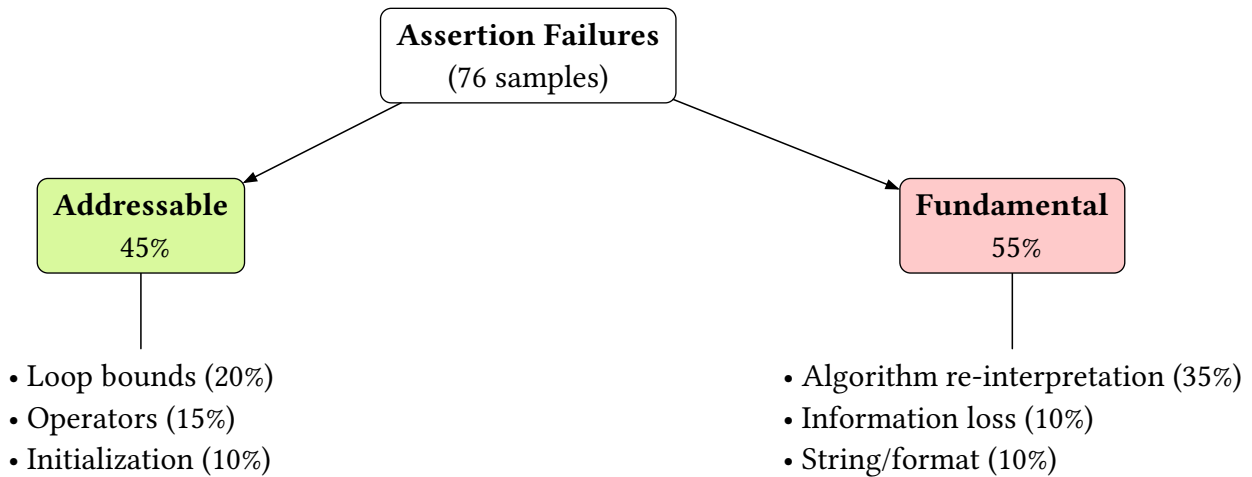


Figure 2: Taxonomy of semantic errors in consistently failing samples.

5.3.2 Trainability Assessment

The error patterns divide into two categories with different implications for further improvement:

Potentially addressable (~45%): Loop bound errors, operator errors, and initialization errors follow systematic patterns that could be targeted through additional training data or error-specific fine-tuning.

Fundamental limitations (~55%): Decompiler information loss cannot be addressed by any model improvement; the information simply does not exist in the input. Algorithm re-interpretation errors require semantic understanding beyond pattern matching; when pseudocode admits multiple valid interpretations, the model cannot determine which matches the test harness without additional context.

This distribution suggests that while targeted interventions may yield incremental improvements, substantial gains in functional correctness likely require either improved decompilation input quality or advances in model reasoning capabilities.

5.4 Error-Specific Fine-Tuning Results

Building on the error analysis, additional LoRA variants were trained to evaluate whether targeted training can address the identified failure patterns.

5.4.1 Synthetic Error Data Generation

A synthetic error dataset was generated using rule-based transformations. The script applies systematic corruptions to clean C code, simulating the semantic errors observed in model failures:

- **Loop bound errors:** Off-by-one transformations such as $i < n \rightarrow i \leq n$ or $i < n - 1 \rightarrow i < n$
- **Operator swaps:** Comparison operator inversions such as $< \rightarrow >$ or $== \rightarrow !=$
- **Initialization errors:** Removal or corruption of variable initializations

Synthetic training pairs were created from 30% of the original ExeBench training data (approximately 1,200 samples), with each sample transformed from clean code into a corrupted variant paired with its original as the target output.

5.4.2 Targeted Error Data

An initial experiment trained a LoRA adapter exclusively on the synthetic error examples, testing whether error correction can be learned in isolation from general decompilation patterns.

The targeted LoRA improves Pass@1 by 4.63 percentage points over baseline, demonstrating that the synthetic error data teaches meaningful correction patterns. However, the compile rate remains nearly unchanged at 21.06%, far below the general LoRA’s 84.33%.

This reveals a critical limitation: training on only 1,200 synthetic examples (even with Ghidra-style artifacts) provides insufficient coverage of the diverse patterns present in real decompiler output. The synthetic transformations are rule-based approximations that may not capture the full complexity of actual Ghidra pseudocode. Additionally, the smaller dataset size compared to the 4,000-sample general training set limits the model’s ability to generalize.

5.4.3 Mixed Training Data

Based on the limited success of targeted training, a second approach combined synthetic error examples with original training data. The mixing ratio was determined by sampling original examples at twice the count of synthetic examples, yielding an approximate 1:2

ratio of error-targeted to general samples. This weighting aims to reinforce error correction while retaining broad decompilation capability.

The mixed LoRA achieves the highest Pass@1 of 28.08%, representing an 81% relative improvement over baseline and a 17% relative improvement over the general LoRA. However, the compile rate decreases from 84.33% to 64.50%.

This trade-off reveals an important insight: the error-focused training data successfully improves semantic correctness (the model produces functionally correct code more often) but at the cost of introducing some syntactic errors. Combining error examples with general training data proves more effective than either approach alone, suggesting that robust decompilation requires both broad pattern coverage and targeted error correction.

5.4.4 Summary

Model	Pass@1 (%)	Compile (%)	Δ Pass@1
Baseline	15.50 $\pm$ 1.08	18.94 $\pm$ 0.53	—
Targeted LoRA	20.13 $\pm$ 0.32	21.06 $\pm$ 0.26	+4.63
General LoRA	23.95 $\pm$ 1.35	84.33 $\pm$ 1.19	+8.45
Mixed LoRA	28.08 $\pm$ 1.30	64.50 $\pm$ 1.54	+12.58

Table 7: Summary of all LoRA configurations

The results demonstrate a clear progression: targeted training alone provides modest semantic improvement but neglects syntactic learning; general training dramatically improves syntax but plateaus on semantics; mixed training achieves the best functional correctness by combining both objectives. The compile rate trade-off in the mixed approach suggests future work could explore curriculum learning [48] or multi-objective optimization to balance syntactic and semantic improvements.

Having established the effectiveness and limitations of LoRA fine-tuning, we now examine whether Knowledge Editing offers an alternative path to targeted correction.

Beyond LoRA fine-tuning, we also investigated whether Knowledge Editing could provide a more surgical approach to correcting specific decompilation errors.

5.5 Knowledge Editing Results

To empirically validate the theoretical limitations of Knowledge Editing, ROME edits were applied to the base CodeLlama model targeting common Ghidra artifacts. Three edit requests mapped type annotations to their C equivalents (undefined4 $\rightarrow$ int, undefined1 $\rightarrow$ char, undefined8 $\rightarrow$ long) and one targeted string literal references (`_LC0`). The edited model was evaluated on the same HumanEval-Decompile test set.

Model	Pass@1 (%)	Compile (%)
Baseline	15.50 $\pm$ 1.08	18.94 $\pm$ 0.53
ROME-edited	15.23 $\pm$ 0.66	22.19 $\pm$ 0.99

Table 8: Comparison of baseline and ROME-edited model performance (2 runs)

The ROME-edited model shows no meaningful improvement in functional correctness, with Pass@1 of 15.23% compared to the baseline’s 15.50%; well within the margin of error. The compile rate increases marginally from 18.94% to 22.19%, suggesting the edits may help avoid some type annotation artifacts. However, this modest syntactic improvement does not translate to semantic gains.

These results confirm the theoretical analysis: the base model already possesses the relevant factual knowledge; when directly asked “What C type does undefined4 represent?”, it correctly responds “int”. The decompilation failures stem not from missing knowledge but from inconsistent application of known facts during generation. ROME, designed to update discrete factual associations, cannot address this reasoning challenge.

6 Discussion

This chapter interprets the experimental findings, examines their implications for neural decompilation, and acknowledges the limitations of this work.

6.1 Interpretation of Results

The experimental results reveal a fundamental distinction between syntactic and semantic aspects of the decompilation task. LoRA fine-tuning achieves a $4.5\times$ improvement in compile rate (from 18.94% to 84.33%) but only a 54% relative improvement in Pass@1 (from 15.50% to 23.95%). This asymmetry suggests that the two challenges require different capabilities: syntactic correction involves learning systematic transformations that map Ghidra artifacts to valid C constructs, while semantic correctness requires reasoning about program logic that may be ambiguous or underspecified in the decompiled representation.

The failure of Knowledge Editing provides insight into the nature of decompilation errors. ROME and similar methods are designed to update atomic factual associations, effectively changing what the model “knows”. However, the base CodeLlama model already possesses the factual knowledge; it correctly identifies that `undefined4` corresponds to `int` when asked directly. The decompilation failures stem not from missing knowledge but from inconsistent application of known facts during generation. This distinction is crucial: neural decompilation is a holistic reasoning task requiring context-dependent inference, not a factual retrieval task susceptible to surgical weight modifications.

The error-specific fine-tuning experiments further illuminate this distinction. The targeted LoRA trained exclusively on synthetic error examples improves Pass@1 modestly (+4.63 percentage points) while leaving compile rate essentially unchanged. This indicates that error correction patterns can be learned, but in isolation they are insufficient; the model requires exposure to the full diversity of decompiler output to generate syntactically valid code. The mixed LoRA, combining error examples with general training data, achieves the highest Pass@1 (28.08%) but at the cost of reduced compile rate (64.50%). This trade-off suggests that optimizing for semantic correctness and syntactic validity may involve competing objectives that are difficult to satisfy simultaneously with a single adapter.

6.2 The Syntactic-Semantic Gap

The consistent pattern across all experiments points to what might be termed a **syntactic-semantic gap** in neural decompilation. Syntactic adaptation, learning to produce valid C code rather than pseudocode artifacts, is readily achievable through fine-tuning. Even the smallest LoRA configuration ($r=8$) improves compile rate from 18.94% to 82.38%. This suggests that syntactic patterns are relatively surface-level and can be captured with modest parameter budgets.

Semantic correctness, however, exhibits diminishing returns. Configurations from $r=32$ to $r=128$ all achieve approximately 23-24% Pass@1, indicating a ceiling that additional capacity

cannot overcome. The error analysis provides insight into why: approximately 55% of failures stem from fundamental limitations (decompiler information loss and algorithm re-interpretation) that cannot be addressed through model adaptation alone. When Ghidra produces a stub function or ambiguous pseudocode, no amount of fine-tuning can recover the original semantics.

The remaining 45% of failures involving loop bounds, operators, and initialization represent the addressable portion of semantic errors. The mixed LoRA’s improvement over the general LoRA (+4.13 percentage points) likely comes from better handling of these systematic error patterns. However, this improvement is modest relative to the syntactic gains, suggesting that even “addressable” semantic errors require more sophisticated interventions than pattern-based training can provide.

6.3 Comparison with State-of-the-Art

The results of this work should be contextualized against dedicated neural decompilation systems. LLM4Decompile [34], the current state-of-the-art, achieves 36.71% re-executability on HumanEval-Decompile with -O2 optimization using their 6.7B parameter model trained through full fine-tuning on billions of code tokens. At comparable model scale, our best configuration (mixed LoRA) reaches 28.08% using approximately 4,000 training samples and training only about 2.3% of total parameters.

This performance gap reflects the fundamental tradeoff between adaptation efficiency and task performance. Full fine-tuning dedicates the entire model capacity to decompilation through extensive training, while LoRA preserves the base model’s general capabilities and requires minimal computational resources [4]. Achieving 76% of the state-of-the-art performance with orders of magnitude less training data suggests that parameter-efficient methods offer a viable path for practitioners who lack the resources for full-scale model training.

Furthermore, LoRA’s modularity enables rapid experimentation with different training objectives, as demonstrated by the error-specific fine-tuning experiments. This flexibility may prove valuable for adapting to specific decompilation scenarios, such as particular compiler versions or optimization levels, without retraining an entire model.

6.4 Implications for Neural Decompilation

These findings have several implications for future work in neural decompilation.

First, the effectiveness of LoRA for syntactic correction suggests that pseudocode-to-code translation could serve as a preprocessing step, converting decompiler output into compilable C before applying more sophisticated semantic analysis. This decomposition might allow specialized models or techniques to focus on each subproblem independently.

Second, the limitations of Knowledge Editing indicate that neural decompilation errors are not localized faults that can be patched individually. Improving semantic correctness likely

requires architectural innovations that enhance reasoning capabilities, such as chain-of-thought prompting [49], retrieval augmentation [50], or multi-pass refinement strategies.

Third, the trade-off observed in mixed training suggests that multi-objective optimization or curriculum learning [48] approaches might better balance syntactic and semantic improvements. Rather than training on a fixed mixture of examples, adaptive strategies could emphasize different objectives at different training stages.

6.5 Limitations

Several limitations constrain the generalizability of these findings.

Dataset scale: The training corpus comprises approximately 4,000 samples from ExeBench, which may be insufficient to capture the full diversity of real-world decompilation scenarios. Larger datasets might shift the observed performance ceilings, and the specific patterns learned may not transfer to binaries compiled with different compilers, optimization levels, or architectures.

Model scale: All experiments use CodeLlama-7B, a relatively small model by contemporary standards. Larger models might exhibit different learning dynamics; in particular, the semantic reasoning ceiling might shift with increased model capacity. However, computational constraints precluded experiments with larger models.

Evaluation scope: The HumanEval-Decompile test set contains 151 relatively short functions with clear input-output specifications. Real-world decompilation often involves longer functions, complex data structures, and incomplete specifications. The reported metrics may overestimate performance on more challenging targets.

Optimization level: All binaries were compiled with -O2 optimization. Different optimization levels produce different decompilation challenges; aggressive optimizations like -O3 introduce additional complexity, while unoptimized builds (-O0) might be substantially easier.

Single decompiler: The evaluation focuses exclusively on Ghidra-generated pseudocode. Other decompilers (IDA Pro, angr) produce different output formats and artifacts, and findings may not transfer directly.

Despite these limitations, the core finding, that syntactic correction is achievable through fine-tuning while semantic reasoning remains challenging, likely holds across settings. The specific performance numbers should be interpreted as indicative rather than definitive.

7 Conclusion

This thesis investigated two adaptation approaches for improving neural decompilation: Low-Rank Adaptation (LoRA) and Knowledge Editing. The research was motivated by the observation that pre-trained language models, while capable of code generation, struggle to produce valid C code from Ghidra-generated pseudocode without task-specific adaptation.

7.1 Summary of Findings

The experimental evaluation yielded several key findings.

First, LoRA fine-tuning substantially improves decompilation performance. The best configuration ($r=64$, $a=64$) achieved a Pass@1 of 23.95%, representing a 54% relative improvement over the baseline of 15.50%. More dramatically, the compile rate improved from 18.94% to 84.33%, demonstrating that fine-tuning effectively teaches the model to generate syntactically valid C code rather than pseudocode artifacts.

Second, Knowledge Editing proved unsuitable for neural decompilation. The investigation revealed that decompilation errors do not stem from missing factual knowledge but from inconsistent application of known transformation rules during generation. This negative result clarifies that neural decompilation is fundamentally a reasoning task, not a knowledge retrieval task.

Third, mixed training combining error-specific examples with general data achieved the highest functional correctness. By combining synthetic error examples with general training data, the mixed LoRA reached 28.08% Pass@1, an 81% relative improvement over baseline. However, this came at the cost of reduced compile rate (64.50%), revealing a trade-off between syntactic and semantic optimization objectives.

Fourth, a “syntactic-semantic gap” characterizes the neural decompilation challenge. Syntactic correction is readily achievable through fine-tuning, with even minimal LoRA configurations dramatically improving compile rates. Semantic correctness, however, exhibits diminishing returns, with approximately 55% of failures stemming from fundamental limitations that model adaptation alone cannot address.

7.2 Contributions

This work makes the following contributions:

1. A systematic comparison of LoRA fine-tuning configurations for neural decompilation, identifying optimal hyperparameters and characterizing the relationship between model capacity and performance.
2. A documented negative result demonstrating the unsuitability of Knowledge Editing for code transformation tasks, with analysis explaining why factual editing methods fail when the underlying challenge is reasoning rather than knowledge.

3. An error taxonomy for neural decompilation failures, distinguishing addressable patterns (loop bounds, operators, initialization) from fundamental limitations (information loss, algorithmic ambiguity).
4. Evidence for the syntactic-semantic gap in neural decompilation, providing a framework for understanding why compile rate improvements do not proportionally translate to functional correctness gains.

7.3 Future Work

Several directions emerge from this research.

The trade-off between syntactic and semantic objectives observed in mixed training suggests that multi-objective optimization or curriculum learning [48] approaches warrant investigation. Adaptive training strategies that balance these objectives throughout the learning process might achieve both high compile rates and improved functional correctness.

The 55% of failures attributed to fundamental limitations highlights the need for improved decompilation input quality. Techniques that preserve more semantic information during decompilation, or that provide additional context such as type information or function signatures, could shift the ceiling on achievable performance.

Finally, the reasoning limitations observed suggest that architectural innovations beyond fine-tuning may be necessary. Chain-of-thought prompting [49], retrieval-augmented generation [50], or multi-pass refinement strategies that decompose the decompilation task into subtasks represent promising directions for future research.

Bibliography

- [1] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, Reprinted, with corr., [36. Druck]. in Addison-Wesley Series in Computer Science. Reading, Mass.: Addison-Wesley, 2002.
- [2] National Security Agency, “Ghidra.” 2019.
- [3] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code.” arXiv, 2023. doi: 10.48550/ARXIV.2308.12950.
- [4] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” no. arXiv:2106.09685. arXiv, Oct. 2021. doi: 10.48550/arXiv.2106.09685.
- [5] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and Editing Factual Associations in GPT,” no. arXiv:2202.05262. arXiv, Jan. 2023. doi: 10.48550/arXiv.2202.05262.
- [6] J. Kirkpatrick *et al.*, “Overcoming Catastrophic Forgetting in Neural Networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017, doi: 10.1073/pnas.1611835114.
- [7] Hex-Rays, “IDA Pro.”
- [8] Y. Shoshitaishvili *et al.*, “SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*, San Jose, CA: IEEE, May 2016, pp. 138–157. doi: 10.1109/SP.2016.17.
- [9] A. Vaswani *et al.*, “Attention Is All You Need,” no. arXiv:1706.03762. arXiv, Aug. 2023. doi: 10.48550/arXiv.1706.03762.
- [10] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [11] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” no. arXiv:1508.07909. arXiv, Jun. 2016. doi: 10.48550/arXiv.1508.07909.
- [12] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” no. arXiv:2307.09288. arXiv, Jul. 2023. doi: 10.48550/arXiv.2307.09288.
- [13] N. Shazeer, “GLU Variants Improve Transformer,” no. arXiv:2002.05202. arXiv, Feb. 2020. doi: 10.48550/arXiv.2002.05202.
- [14] M. Geva, R. Schuster, J. Berant, and O. Levy, “Transformer Feed-Forward Layers Are Key-Value Memories,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 5484–5495. doi: 10.18653/v1/2021.emnlp-main.446.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” no. arXiv:1512.03385. arXiv, Dec. 2015. doi: 10.48550/arXiv.1512.03385.

- [16] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” no. arXiv:1607.06450. arXiv, Jul. 2016. doi: 10.48550/arXiv.1607.06450.
- [17] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” no. arXiv:1910.07467. arXiv, Oct. 2019. doi: 10.48550/arXiv.1910.07467.
- [18] J. Wei *et al.*, “Emergent Abilities of Large Language Models,” no. arXiv:2206.07682. arXiv, Oct. 2022. doi: 10.48550/arXiv.2206.07682.
- [19] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” 2018.
- [20] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models,” no. arXiv:2001.08361. arXiv, Jan. 2020. doi: 10.48550/arXiv.2001.08361.
- [21] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” no. arXiv:2302.13971. arXiv, Feb. 2023. doi: 10.48550/arXiv.2302.13971.
- [22] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” no. arXiv:2104.09864. arXiv, Nov. 2023. doi: 10.48550/arXiv.2104.09864.
- [23] J. Wei *et al.*, “Finetuned Language Models Are Zero-Shot Learners,” no. arXiv:2109.01652. arXiv, Feb. 2022. doi: 10.48550/arXiv.2109.01652.
- [24] C. Fu *et al.*, “A Neural-based Program Decompiler,” no. arXiv:1906.12029. arXiv, Jun. 2019. doi: 10.48550/arXiv.1906.12029.
- [25] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP,” no. arXiv:1902.00751. arXiv, Jun. 2019. doi: 10.48550/arXiv.1902.00751.
- [26] X. L. Li and P. Liang, “Prefix-Tuning: Optimizing Continuous Prompts for Generation,” no. arXiv:2101.00190. arXiv, Jan. 2021. doi: 10.48550/arXiv.2101.00190.
- [27] B. Lester, R. Al-Rfou, and N. Constant, “The Power of Scale for Parameter-Efficient Prompt Tuning,” no. arXiv:2104.08691. arXiv, Sep. 2021. doi: 10.48550/arXiv.2104.08691.
- [28] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” no. arXiv:2305.14314. arXiv, May 2023. doi: 10.48550/arXiv.2305.14314.
- [29] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning, “Fast Model Editing at Scale,” no. arXiv:2110.11309. arXiv, Jun. 2022. doi: 10.48550/arXiv.2110.11309.
- [30] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, “Memory-Based Model Editing at Scale,” no. arXiv:2206.06520. arXiv, Jun. 2022. doi: 10.48550/arXiv.2206.06520.
- [31] C. Zheng *et al.*, “Can We Edit Factual Knowledge by In-Context Learning?,” no. arXiv:2305.12740. arXiv, May 2023. doi: 10.48550/arXiv.2305.12740.
- [32] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” no. arXiv:2107.03374. arXiv, Jul. 2021. doi: 10.48550/arXiv.2107.03374.

- [33] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A Method for Automatic Evaluation of Machine Translation,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, Philadelphia, Pennsylvania: Association for Computational Linguistics, 2001, p. 311. doi: 10.3115/1073083.1073135.
- [34] H. Tan, Q. Luo, J. Li, and Y. Zhang, “LLM4Decompile: Decompiling Binary Code with Large Language Models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 3473–3487. doi: 10.18653/v1/2024.emnlp-main.203.
- [35] W. K. Wong *et al.*, “Refining Decompiled C Code with Large Language Models,” no. arXiv:2310.06530. arXiv, Nov. 2023. doi: 10.48550/arXiv.2310.06530.
- [36] P. Hu, R. Liang, and K. Chen, “DeGPT: Optimizing Decompiler Output with LLM,” in *Proceedings 2024 Network and Distributed System Security Symposium*, San Diego, CA, USA: Internet Society, 2024. doi: 10.14722/ndss.2024.24401.
- [37] N. Jiang *et al.*, “Nova: Generative Language Models for Assembly Code with Hierarchical Attention and Contrastive Learning,” no. arXiv:2311.13721. arXiv, Nov. 2025. doi: 10.48550/arXiv.2311.13721.
- [38] X. Li *et al.*, “Model Editing for LLMs4Code: How Far Are We?,” no. arXiv:2411.06638. arXiv, Dec. 2024. doi: 10.48550/arXiv.2411.06638.
- [39] J. Armengol-Estapé, J. Woodruff, A. Brauckmann, J. W. D. S. Magalhães, and M. F. P. O’Boyle, “ExeBench: An ML-scale Dataset of Executable C Functions,” in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, San Diego CA USA: ACM, Jun. 2022, pp. 50–59. doi: 10.1145/3520312.3534867.
- [40] GNU Project, “GNU Compiler Collection.”
- [41] D. Han, M. Han, and Unsloth team, “Unsloth.” 2023.
- [42] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” no. arXiv:1912.01703. arXiv, Dec. 2019. doi: 10.48550/arXiv.1912.01703.
- [43] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” no. arXiv:1711.05101. arXiv, Jan. 2019. doi: 10.48550/arXiv.1711.05101.
- [44] Google Research, “Google Colaboratory.”
- [45] Baden-Württemberg Ministerium für Wissenschaft, Forschung und Kunst, “bwHPC.”
- [46] P. Wang *et al.*, “EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models.” arXiv, 2023. doi: 10.48550/ARXIV.2308.07269.
- [47] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” no. arXiv:1904.09751. arXiv, Feb. 2020. doi: 10.48550/arXiv.1904.09751.
- [48] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum Learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning*, Montreal Quebec Canada: ACM, Jun. 2009, pp. 41–48. doi: 10.1145/1553374.1553380.

- [49] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” no. arXiv:2201.11903. arXiv, Jan. 2023. doi: 10.48550/arXiv.2201.11903.
- [50] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” no. arXiv:2005.11401. arXiv, Apr. 2021. doi: 10.48550/arXiv.2005.11401.